

TP — Anonymisation des données personnelles

Contexte : vous travaillez pour Traiteur-GPT, une entreprise de restauration événementielle. Vos collègues collectent des données clients (noms, téléphones, emails, adresses, coordonnées bancaires) dans des messages texte libres. Votre mission est d'explorer et d'améliorer le système d'anonymisation mis en place.

Prérequis : démarrer l'application avec `bash start.sh` puis ouvrir <http://localhost:8000>

Question 1 — Concepts RGPD (2 pts)

Quelle est la différence fondamentale entre **anonymisation** et **pseudonymisation** au sens du RGPD ?

Pour chacune des 4 méthodes de l'application (Presidio, Scrubadub, Faker, Caviardage), indiquez dans quelle catégorie elle se situe et justifiez en une phrase.

Question 2 — Réversibilité (1 pt)

La pseudonymisation Faker affiche une **table de correspondance** dans l'interface.

a) À quoi sert cette table dans un contexte professionnel réel ? b) Où faudrait-il stocker cette table pour que le système soit conforme au RGPD ? Citez deux exigences concrètes.

Note — Limites de Faker à grande échelle

Faker puise ses noms dans un répertoire fini : environ 500 prénoms × 500 noms de famille pour la locale `fr_FR`, soit ~250 000 combinaisons possibles. Dès que la base dépasse quelques dizaines de milliers de lignes, le **paradoxe des anniversaires** entre en jeu : la probabilité qu'un pseudonyme soit attribué à deux personnes réelles différentes devient significative — et quasi-certaine au-delà du million de lignes.

Conséquence concrète : deux clients distincts, par exemple *Sophie Martin* et *Jean Dupont*, peuvent tous les deux se voir attribuer `Thomas Bernard`. La table de correspondance contient alors deux entrées qui pointent vers le même pseudonyme, ce qui rend la réidentification ambiguë (on ne sait plus qui est derrière `Thomas Bernard`).

Pour des volumes industriels, on préfère des pseudonymes **déterministes et sans collision** :

- **UUID v4** : identifiant aléatoire de 128 bits — le risque de collision est astronomiquement faible.
- **HMAC-SHA256(clé secrète, identité réelle)** : pseudonyme déterministe (la même personne donne toujours le même pseudonyme) et irréversible sans la clé, ce qui simplifie la table de correspondance.
- **Compteur incrémental** : `CLIENT_000001`, `CLIENT_000002` ... — trivial, mais lisible et sans collision.

Ces approches sacrifient la vraisemblance du pseudonyme (un UUID ne ressemble pas à un prénom) au profit de la robustesse à l'échelle.

Peut-on configurer Faker pour éviter les doublons ? Oui, partiellement : Faker expose un proxy `.unique` qui mémorise les valeurs déjà générées et réessaie automatiquement jusqu'à en trouver une nouvelle :

```
fake.unique.name() # garantit qu'aucun nom ne sera répété dans cette session
fake.unique.clear() # remet le compteur à zéro
```

Dès que le pool est épuisé (~250 000 noms pour `fr_FR`), Faker lève une `UniquenessException` plutôt que de boucler indéfiniment. Cette garantie ne vaut que pour une instance Faker donnée : si le processus redémarre ou si plusieurs workers tournent en parallèle, il n'y a plus de coordination entre eux et les collisions redeviennent possibles.

Question 3 — Lecture de code : ordre de traitement (2 pts)

Ouvrez `main.py` et lisez la fonction `redact_text()` (ligne ~265).

Elle applique d'abord trois expressions régulières **avant** d'appeler Presidio.

a) Pourquoi ce choix architectural ? Dans quel cas cela est-il nécessaire ? b) Identifiez les trois types de données traités par ces regex et donnez un exemple de texte que Presidio pourrait manquer mais que la regex capturerait.

Question 4 — API REST (1 pt)

Depuis un terminal, exécutez la commande suivante :

```
curl http://localhost:8000/status
```

a) Que retourne cet endpoint ? b) Expliquez comment l'interface web l'utilise (indice : cherchez `loadStatus` dans `index.html`).

Question 5 — Scénario métier (2 pts)

Le responsable logistique doit transmettre les commandes clients à un prestataire de livraison externe. Ce prestataire a besoin du nom et de l'adresse pour effectuer la livraison, mais Traiteur-GPT doit pouvoir contester une erreur de livraison en retrouvant l'identité réelle du client.

Quelle méthode recommandez-vous ? Décrivez le flux de données entre Traiteur-GPT et le prestataire. Quel cadre légal RGPD encadre ce partage de données avec un tiers externe, et quelle obligation contractuelle cela implique-t-il ?

Question 6 — Lecture de code : remplacement inversé (2 pts)

Dans la fonction `faker_pseudonymize()` , les résultats de détection sont triés ainsi avant le remplacement :

```
results = sorted(_analyze(text, detect_health), key=lambda r: r.start, reverse=True)
```

a) Pourquoi trier dans l'ordre **décroissant** (de la fin vers le début du texte) ? b) Écrivez un exemple de texte avec deux entités et montrez ce qui se passerait si on triait dans l'ordre croissant.

Question 7 — Biais des modèles NLP (1 pt)

Les modèles spaCy sont entraînés principalement sur des corpus de presse francophone et Wikipédia. Testez successivement ces trois textes dans l'application :

```
Texte A : Bonjour, je suis Jean Dupont, mon email est jean.dupont@gmail.com  
Texte B : Bonjour, je suis Aminata Kouyaté, mon email est a.kouyate@gmail.com  
Texte C : Bonjour, je suis Chidinma Okonkwo, mon email est c.okonkwo@gmail.com ← nom igbo (Nigeria)
```

a) Presidio détecte-t-il ces trois noms avec la même fiabilité ? Observez attentivement ce qui est caviardé (ou non) pour chaque texte, et formulez une hypothèse sur la cause des différences. b) Quelle

conséquence concrète cela a-t-il pour une entreprise qui utilise ce système pour protéger l'ensemble de ses données clients ?

Question 8 — Robustesse et fallback (1 pt)

Dans `main.py`, au démarrage, le code tente d'initialiser Presidio avec le modèle français selon cette priorité : `fr_core_news_lg` → `fr_core_news_md` → `fr_core_news_sm`.

a) Que se passe-t-il si aucun modèle français n'est trouvé ? b) Quelle conséquence cela a-t-il sur la qualité de la détection pour des textes en français ?

Question 9 — Frontend : debounce (1 pt)

Dans `index.html`, l'appel API est enveloppé dans un debounce de 320 ms :

```
const debouncedProcess = debounce(processText, 320);
document.getElementById("userInput").addEventListener("input", e => {
  debouncedProcess(e.target.value);
});
```

a) Expliquez le rôle du debounce. b) Quelles seraient les conséquences (côté serveur et côté utilisateur) si on remplaçait `320` par `0` ?

Question 10 — Extension du code (3 pts)

Ajoutez une cinquième méthode d'anonymisation : la **suppression pure**. Les entités détectées sont supprimées du texte (retirées sans remplacement, en laissant un espace simple).

Fournissez :

1. La fonction Python `deletion_anonymize(text: str, detect_health: bool = False) -> dict` à ajouter dans `main.py`
 2. La modification de la route `POST /anonymize` pour inclure la clé `"deletion"` dans la réponse (en propageant le paramètre `detect_health`)
 3. (Bonus) Les modifications HTML/JS minimales pour afficher cette cinquième colonne dans l'interface
-

Question 11 — PatternRecognizer et données Art. 9 (2 pts)

L'application propose une option "Détecter les données de santé (Art. 9 RGPD)" activable via une case à cocher sous la barre d'exemples.

- a) Activez l'option et soumettez l'exemple  **Données de santé (Art. 9)**. Listez les entités de type `HEALTH` détectées. Pourquoi ces données relèvent-elles de l'article 9 du RGPD plutôt que de l'article 6 ?
- b) Dans `main.py`, repérez le paramètre `context` passé au `MedicalTermRecognizer`. Quel est son rôle ?
Donnez un exemple concret de phrase où sa présence ferait augmenter le score de confiance par rapport à une phrase sans ce mot de contexte.
- c) Citez **deux limites** de cette approche par liste de mots-clés (`deny_list`) pour détecter des données de santé exprimées en langage naturel. Pour chaque limite, proposez une technologie alternative.
-

Durée indicative : 2h30 — Barème : /18 pts (+ 2 pts bonus)